Colour Maximite 2

App User Guide

The programs that come with the Colour Maximite 2

For firmware 7.00.00RC5 and later

Peter Mather

(10 November 2026)

The firmware, the programs and this guide are at
https://github.com/UKTailwind/CMM2V1.5

About This Guide

This guide is for anyone using the programs that come with the Colour Maximite 2: MicroPython, Doom, Heretic, Quake, ScummVM, the NES, ZX Spectrum, Amstrad CPC, Commodore 64, Atari 8-bit, Apple //e and Amiga 500 emulators, the 386 PC, the C compiler, the VT100 terminal and zeptoforth. It covers installing each one, starting it, its keys and leaving it. It assumes you can use the MMBasic command prompt and copy files to the SD card.

Support

Questions should be raised on the Back Shed forum (http://www.thebackshed.com/forum/Microcontrollers), and problems with the programs reported as issues of the repository above.

Copyright

This guide is copyright 2026 Peter Mather and is distributed under a Creative Commons Attribution-NonCommercial-ShareAlike 3.0 Australia license (CC BY-NC-SA 3.0), as the user manual is. The programs have their own licences (see Appendix A).

Contents

Introduction 5
What Is Here 5
Installing the Programs 5
Starting a Program 5
Leaving a Program 6
What the Programs Use 6
If a Program Will Not Start 6
MicroPython 7
Starting It 7
Using It 7
The Keys 7
Your Files 7
What It Has 7
Doom and Heretic 8
Doom 8
Heretic 8
The Keys 8

Quake 9
The Keys 9
The Mouse 9
The Mouse Wheel 9
Settings 10
Your Own Copy of Quake 10
ScummVM 11
Starting It 11
The Keys 11
Saving Your Game 11
The Games That Come With It 12
Adding Games You Own 12
The NES Emulator 13
Starting It 13
The Controls 13
If a Gamepad Does Nothing 13
The ZX Spectrum Emulator 14
Starting It 14
Loading Games 14
The Keys 14
The Amstrad CPC Emulator 15
Starting It 15
Loading Games 15
The Keys 15
The Commodore 64 Emulator 16
Starting It 16
Loading Programs 16
The Keys 16
Joysticks and the Mouse 16
The Atari 8-bit Emulator 18
Starting It 18
The ROMs 18
BASIC 18
Loading Programs 18
The Keys 18
Joysticks, Paddles and the Mouse 19
Known Problems 19
The Apple //e Emulator 20
Starting It 20
Disks 20
The Keys 20
Joysticks 21
The Serial Console 21
The Amiga 500 Emulator 22
What You Need 22

Starting It....22
The Display....23
The Keys....24
Saving Your Work....24
The Mouse and Joysticks....24
Leaving It....24
What Has Been Tried....24
Licences....25
The 386 PC....26
Starting It....26
Leaving It....26
The Keys....26
The Mouse and Sound....26
Making It Faster....26
Moving a PC....26
The C Compiler....27
Installing It....27
When It Is Worth Using....27
Compiling an MMBasic Program....27
Compiling a C Program....27
CC's Options....28
How Much Faster....28
When Things Go Wrong....29
The VT100 Terminal....30
Connecting....30
Starting It....30
Leaving It....30
Using It With a PicoMite....30
The Keys....30
The Setup Menu....31
Moving Files With XMODEM....31
zeptoforth....32
Why Forth....32
Starting It....32
Leaving It....32
The Console....32
Editing Files....32
Loading a Program....33
Keeping Your Work....33
Blocks....33
Graphics, Sound and I/O....33
Double Precision....33
Learning More....34
Errors....34
Memory....34
Appendix A: Licences....35

Introduction

Your Colour Maximite 2 can run much more than MMBasic. This guide covers the programs that come with the firmware: games, emulators, a C compiler, MicroPython, Forth and a terminal. It tells you how to install each one, start it, use it and leave it.

You do not need to be a programmer to use any of them, apart from the C compiler, MicroPython and zeptoforth. If you want to write programs of this kind yourself, that is what the companion manual *Colour Maximite 2 Loadable Programs* is for.

What Is Here

Program	What it does	Folder
MicroPython	Python, with the screen, sound and I/O pins	A:/
Doom	The classic game, with the free Freedoom game data	A:/Doom
Heretic	Raven Software's game, with its shareware episode	A:/heretic
Quake	id Software's game, with its shareware episode	A:/quake
ScummVM	Classic adventure games from LucasArts, Sierra and others	A:/quest
NES	A Nintendo Entertainment System emulator, with two free games	A:/nes
ZX Spectrum	A 48K and 128K Spectrum emulator	A:/spectrum
Amstrad CPC	A 464, 664 and 6128 emulator	A:/cpc
Commodore 64	A C64 emulator, with its 1541 disk drive	A:/c64
Atari 8-bit	The Atari 400/800, XL/XE and 5200 emulator atari800	A:/atari800
Apple //e	An Apple //e emulator, with two Disk II drives	A:/apple2
Amiga 500	An Amiga 500 emulator, with your own Kickstart and disks	A:/amiga
386 PC	A PC that runs FreeDOS, MS-DOS, Windows 3.1 and Windows 95	A:/pc386
C compiler	Compiles C and MMBasic programs on the machine itself	A:/cc
VT100 terminal	Makes the Colour Maximite 2 the screen and keyboard of a PicoMite or other computer	A:/vt100
zeptoforth	A Forth with tasks, double precision and a screen editor	A:/zeptoforth

All of them run on both the Generation 1 and the Generation 2, from the same files, except zeptoforth, which has a file for each, and Quake and the Amiga, which need a Generation 2.

Installing the Programs

The programs are in the file CMM2_Programs_7.00.00RC5.zip, which is posted with the firmware.

1. Update the firmware first. Some of these programs need the firmware of the same release.

2. Unzip the file on a PC.

3. Copy everything in it to the root of the SD card, keeping the folders. Use a card reader: some of the files are tens of megabytes, which would take many minutes over the serial console.

You need only copy the folders you want. If you do not want Doom, leave out the Doom folder. Keep the licences folder with the programs if you pass them on to anyone else.

Starting a Program

A program is a file ending .app, and MMBasic runs it with RUN. Change to its folder first, as most of them look there for their files and keep their settings there:

```
CHDIR "A:/nes"
RUN "nes.app"
```

Words after a comma are passed to the program:

```
RUN "scummvm.app", dott
```

Do not put them in quotes. Each chapter of this guide says what its program accepts.

A BASIC program can start one too. MMBasic carries on with the next statement when it ends, with the program's variables as they were.

Leaving a Program

Programs differ in how you leave them, as an emulator has to pass most keys on to the machine it is emulating:

Program	How to leave it
MicroPython	machine.reset() at its prompt
Doom, Heretic, Quake	Ctrl-C
ScummVM	Ctrl-C. In a game, Ctrl+Alt+Del goes back to the list of games
NES	Ctrl+Alt+End, or Ctrl-C
ZX Spectrum, Amstrad CPC, Commodore 64, Atari 8-bit, Apple //e	Ctrl+Alt+End (F9 too on the Atari). Ctrl-C is a key of the emulated machine
386 PC	Ctrl+Alt+End. See its chapter first, as this switches the PC off
VT100 terminal	Shift-F12, then Quit. Ctrl-C goes to the computer at the other end

When a program ends, MMBasic tidies up after it: its files are closed, its sound is stopped, its I/O pins are released, and the video mode, font, colours and palette are put back.

What the Programs Use

The keyboard and mouse. All the programs use a USB keyboard. Those that use a mouse take a USB mouse, or a PS/2 mouse set up with OPTION MOUSE.

USB gamepads. The NES, Spectrum, CPC, C64, Atari and Apple //e emulators use them. A DualShock 3 sends nothing until its PS button has been pressed. A generic pad is usually a low speed device, and has to go straight into the Colour Maximite 2, not through a hub.

The screen. A program sets the video mode it needs, and MMBasic puts yours back when it ends. Your monitor may take a moment to lock on each time.

Memory. A Generation 2 keeps 8MB of its memory for these programs. A Generation 1 lends a program the largest free part of MMBasic's memory while it runs, and some programs borrow the display memory too, so on a Generation 1 the screen can change as a program starts.

If a Program Will Not Start

Message	What it means
Needs a newer firmware	The program needs a later version of the firmware than you have. Update it.
Not a loadable program	The file is not a program, or is not the one you meant.
The program file is damaged	Copy it to the card again.
Needs a G2	The program needs a Generation 2. Of the programs in this guide, only Quake does.
Needs n KB, n KB free in one block	There is not enough memory. Restart the Colour Maximite 2 and try it from the command prompt, not from inside a BASIC program. On a Generation 1, OPTION RAM leaves less: use OPTION FLASH.

If a program cannot find its files, you have probably started it from the wrong folder. CHDIR to its folder and try again.

MicroPython

MicroPython is the Python language for small computers. This is the MicroPython of the Pico Computer 3, so programs written for that run here too.

Starting It

micropython.app goes in the root of the card. At the MMBasic prompt:

```
RUN "A:/micropython.app"
```

It runs boot.py and main.py if they are in the current folder, and then shows its >>> prompt. To run one program and go straight back to MMBasic, name it:

```
RUN "A:/micropython.app", myprog.py
```

Using It

The prompt appears on the screen, and on the serial console. Type Python at it:

```
>>> print("Hello", 6 * 7)
Hello 42
```

These are ready at the prompt as it starts:

Name	What it does
ls(), cd(), cat()	List a folder, change folder, show a file
edit()	The editor
run()	Run a program
fm()	The file manager
plot(), Turtle()	Drawing and turtle graphics
play(), tone()	Sound
draw_jpg()	Show a JPEG picture
keydown(), mouse()	Read the keyboard and the mouse

The Keys

Keys	What they do
Ctrl-C	Cancels a line, or stops a running program
Ctrl-D	A soft reset, which starts MicroPython again

To leave MicroPython and go back to MMBasic, type:

```
import machine
machine.reset()
```

Your Files

The SD card is at / and also at /sd. A USB flash drive is at /usb when one is plugged in. Modules of your own can go in /lib.

What It Has

- The Pico Computer 3's modules for the display, the keyboard, the mouse, sound and pictures.
- SQLite, the database, and ulab, for arrays and fast numerical work.
- The machine module, for the rear I/O pins, by the PicoMite's GP numbers.

Python runs about twice as fast here as it does on the RP2350.

Doom and Heretic

Both are the original games, running at full speed on either generation.

Doom

```
CHDIR "A:/Doom"
RUN "doom.app"
```

This plays Freedoom Phase 1, a free replacement for Doom's game data. The music is played on the firmware's FM synthesiser.

Always start it from its folder, so that its settings and saved games are kept there.

Your Own Copy of Doom

If you have Doom's own game data, there are two ways to use it:

- Copy it into A:/Doom as doom.wad or doom1.wad, and remove freedoom1.wad.
- Name it on RUN's line:

```
RUN "doom.app", -iwad x.wad
```

Heretic

```
CHDIR "A:/heretic"
RUN "heretic.app"
```

The folder holds the whole of Heretic's shareware version 1.2, the first episode, unchanged. HERETIC1.WAD is what the game plays, and the DOS programs beside it are not used here.

If you have the full game, put its heretic.wad in the folder and it is used in place of the shareware.

LICENSE.DOC, in the same folder, says how the shareware may be passed on: free of charge, and compressed if it is sent electronically.

The Keys

Action	Doom	Heretic
Move and turn	The arrows	The arrows
Fire	Ctrl	Ctrl
Open, use	Space	Space
Run	Shift	Shift
Strafe	Alt with the arrows, or , and .	Alt with the arrows, or , and .
The menu	Esc	Esc
Choose an artifact		[and]
Use an artifact		Enter
Leave the game	Ctrl-C	Ctrl-C

Saving and loading are in the game's own menu, under Esc.

Quake

This is id Software's Quake, with its shareware episode. It needs a Generation 2. It draws 320 by 240 in 256 colours, and its timed demo runs at about 53 frames a second with the sound on.

```
CHDIR "A:/quake"
RUN "quake.app"
```

Always start it from its folder. It finds its game data there, and keeps its settings and saved games in the id1 folder inside it.

The folder holds the whole of Quake's shareware version 1.06, unchanged. ID1/PAK0.PAK is what the game plays, and the DOS and Windows programs beside it are not used here. SLICNSE.TXT, in the same folder, says how the shareware may be passed on: free of charge, as a whole, and compressed if it is sent electronically.

Sound effects play, at PLAY VOLUME's level. Quake's music was on its CD, so there is none here, and there is no network play.

The Keys

Action	Key
Move forward and back	Up and Down
Turn	Left and Right
Sidestep	, and . or Alt with Left and Right
Run	Shift
Fire	Ctrl, or the left mouse button
Jump, or swim up	Space or Enter
Look up and down	A and Z
Look straight ahead	End
Choose a weapon	1 to 8, or / for the next one
Show the scores	Tab
The menu	Esc
Help	F1
Save and load	F2 and F3
Quick save and quick load	F6 and F9
The console	The key to the left of 1, called the backquote
Leave the game	F10 then Y, or Ctrl-C

The keys can be changed under Options in the menu.

The Mouse

Moving the mouse turns you. The left button fires, and the right button walks forward. Hold the middle button to look up and down with the mouse as well.

To keep looking with the mouse without holding a button, open the console and type:

```
+mlook
```

The sensitivity is under Options in the menu.

The Mouse Wheel

Quake leaves the wheel unused, and turning it shows a message such as "MWHEELUP is unbound, hit F4 to set". To make it change weapons, open the console and type:

```
bind mwheelup "impulse 10"
bind mwheeldown "impulse 12"
```

Settings

Settings and key changes are saved in id1/config.cfg when you leave the game with F10 or Ctrl-C. If the Colour Maximite 2 is restarted while the game is running, any changes since it started are lost.

Mouse look is not one of the settings saved. To have it on every time, make a text file called autoexec.cfg in A:/quake/id1, which the game runs each time it starts:

```
+mlook
bind mwheelup "impulse 10"
bind mwheeldown "impulse 12"
```

Your Own Copy of Quake

If you have the full game, copy its PAK0.PAK and PAK1.PAK into A:/quake/id1 in place of the shareware's PAK0.PAK. This has not been tried: the program has only been tested with the shareware.

The game has 12MB of MMBasic's memory. If it runs out, ask for more on RUN's line, up to about 15:

```
RUN "quake.app", -mem 15
```

ScummVM

ScummVM plays classic adventure games from their original data files. This version comes from FRANK Quest, rh1tech's port of ScummVM to the RP2350. It has the AGI, SCI, SCUMM (versions 1 to 7), Gobliiins and Kyrandia engines.

Starting It

```
CHDIR "A:/quest"
RUN "scummvm.app"
```

It lists the games it finds in folders of A:/quest. Choose one with the arrow keys and press Enter.

To start a game at once, give its folder's name:

```
CHDIR "A:/quest"
RUN "scummvm.app", dott
```

On a Generation 1 the screen changes to MODE 7 as it starts, as ScummVM borrows the display memory.

The Keys

Keys	What they do
The mouse	Most of these games are played with it
F5	Save and restore (see below)
Esc	Skips a cutscene, in most games
Ctrl+Alt+Del	Back to the list of games
Ctrl-C	Leave ScummVM

Saving Your Game

Saved games are kept in A:/quest/saves. How you save depends on who made the game.

LucasArts Games

This means Monkey Island, Indiana Jones, Day of the Tentacle, Sam & Max, Loom and Full Throttle.

Press **F5** to open the save menu. It lists nine slots, with the date each one was saved.

Keys	What they do
Up, Down or 1 to 9	Choose a slot
S	Save in that slot. If the slot is in use it asks first
R	Restore that slot
Esc or F5	Close the menu

A message tells you when the game has been saved.

There are also shortcuts, which work without the menu and use the same slots:

Keys	What they do
Alt-1 to Alt-9	Save in that slot at once, replacing what was there
Ctrl-1 to Ctrl-9	Restore that slot at once

You cannot save during a cutscene, or before the game has properly started. The menu says so if that is why.

Sierra Games

The AGI and SCI games use their own save and restore screens, as they did on a PC. Press **F5** to save and **F7** to restore, or use the game's menu bar, which Esc opens in most of them. You can choose a slot and give each save a name.

Gobliiins and Kyrandia

These have save and restore in their own menus, on the game's screen.

The Games That Come With It

Folder	Game
freddy	Freddy Pharkas, Frontier Pharmacist: the interactive demo (Sierra, 1993)
agi	DG: The Adventure Game (Robin Gravel, 2001; public domain)
dgc64	Another DG Game: I Want My C64 Back (Robin Gravel, 2002; public domain)

Adding Games You Own

Games you own go in folders of A:/quest. The folder's name tells ScummVM which game it is, so it must be one of those below, in upper or lower case. A folder with any other name is not listed.

Copy the game's data files only, not its .EXE and .DRV files.

LucasArts (SCUMM)

Folder	Game
mi1	The Secret of Monkey Island, VGA floppy (needs 000.LFL)
mi1ega	The Secret of Monkey Island, EGA floppy (needs 000.LFL)
monkey, mi1cd	The Secret of Monkey Island, CD (needs monkey.000)
monkey2, mi2	Monkey Island 2: LeChuck's Revenge (needs monkey2.000)
loom	Loom, VGA (needs 000.LFL)
indy4, atlantis	Indiana Jones and the Fate of Atlantis (needs atlantis.000)
dott, tentacle	Day of the Tentacle (needs tentacle.000)
samnmax, sam	Sam & Max Hit the Road
ft, fulltp	Full Throttle (needs ft.la0)

Sierra (SCI)

Folder	Game
kq1 kq4 kq5 kq6	King's Quest I, IV, V, VI
sq1 sq3 sq4 sq5	Space Quest I, III, IV, V
lsl1 lsl2 lsl3 lsl5 lsl6	Leisure Suit Larry 1, 2, 3, 5, 6
pq1 pq2 pq3	Police Quest I, II, III
qfg1 qfg1vga qfg2 qfg3	Quest for Glory I (EGA, VGA), II, III
laurabow laurabow2	Laura Bow 1 and 2
ecoquest ecoquest2	EcoQuest 1 and 2
iceman longbow camelot	Codename: ICEMAN, Conquests of the Longbow, of Camelot
freddy jones pepper slater	Freddy Pharkas, Jones in the Fast Lane, Pepper's Adventures in Time, Slater & Charlie
castlebrain islandbrain mothergoose	Castle and Island of Dr. Brain, Mixed-Up Mother Goose

Others

Folder	Game
kyra1, kyrandia	The Legend of Kyrandia (needs KYRA.DAT: copy it from A:/quest into the game's folder)
kyra2, handoffate	The Legend of Kyrandia 2: The Hand of Fate (needs KYRA.DAT too)
gob1 gob2 gob3	Gobliiins, Gobliins 2, Goblins 3
agi	One AGI game (version 2) of any kind, such as a fan game

Other games, and demos other than those of Gobliiins and Kyrandia, are not supported. Nor are LucasArts' last two: The Dig (SCUMM version 7, not built into FRANK Quest) and The Curse of Monkey Island (SCUMM version 8, at 640 x 480).

The NES Emulator

This is frank-nes, Mikhail Matveev's NES emulator, on the QuickNES core by Shay Green. It runs at full speed on both generations, in MODE 7,8.

Starting It

```
CHDIR "A:/nes"
RUN "nes.app"
```

Its cartridges are the .nes files in A:/nes. It comes with two free games, Nova the Squirrel and Thwaite. To add more, copy the .nes files into the folder.

The welcome screen moves on by itself after 10 seconds, whether or not a button has worked.

The Controls

It uses USB gamepads and the USB keyboard.

Pad	Keyboard	NES
D-pad or stick	The arrows or WASD	D-pad
Circle, Triangle	X	A
Cross, Square	Z	B
Start	Enter	Start
Select	Space	Select

These work at any time:

Keys	What they do
Start and Select together, or F12	The menu
F11	Back to the cartridges
Ctrl+Alt+End, or Ctrl-C	Back to MMBasic

If a Gamepad Does Nothing

- A DualShock 3 sends nothing until its PS button has been pressed.
- A generic pad is usually a low speed device, and has to go straight into the Colour Maximite 2, not through a hub.
- If a generic pad gives no response after a restart, unplug it and plug it in again.

The ZX Spectrum Emulator

This is pico-spec, DnCraptor's port of the ESPectrum emulator by Victor Iborra and David Crespo. It runs at the Spectrum's 50 frames a second on both generations, in MODE 7,8.

Starting It

```
CHDIR "A:/spectrum"
RUN "spectrum.app"
```

It starts as a 48K Spectrum. The ROMs are built into the program, so it needs nothing else on the card. The F1 menu chooses another machine. The 48K and 128K have been tried here; the others in its menu are as pico-spec has them.

Loading Games

Put tapes and snapshots in A:/spectrum, and open them with **F5**, which starts a tape by itself. You can also choose a tape and type LOAD "" as you would on a real Spectrum.

The program calls the folder it was started in SD:/, and keeps its settings there.

The Keys

Keys	What they do
F1	The menu
F3, F4	Load and save a snapshot, in numbered slots
F5	Open a file
F6	Start or stop the tape
F9, F10	Volume down and up
F11	Reset the Spectrum
F12	Restart the emulator. It asks first, with No chosen
Shift	Caps Shift
Ctrl	Symbol Shift
Ctrl+Alt+End	Back to MMBasic

Ctrl-C is a Spectrum key here, Symbol Shift and C, so it does not end the program.

Snapshots are how you save your place in a game: F4 saves one and F3 loads it.

USB gamepads step through the emulator's menus, and are the Spectrum's joysticks. They are Kempston joysticks unless the menu says otherwise.

The Amstrad CPC Emulator

This is frank-cpc, rh1tech's port of the Caprice32 emulator. It runs at the CPC's 50 frames a second on both generations, in MODE 8,8.

Starting It

```
CHDIR "A:/cpc"
RUN "cpc.app"
```

It starts as a CPC 6128. Press a key to clear its welcome screen. **F12** chooses the 464, 664 or 6128, and 64K, 128K or 576K of memory. Its ROMs are in its rom folder.

The program calls the folder it was started in /cpc, and keeps its settings there in cpc.ini.

Loading Games

Put disks (.dsk), tapes (.cdt, .cas) and cartridges (.cpr) in A:/cpc/disk, and open them with **F11**.

- For a tape on the 664 or 6128, type |TAPE first.
- Leave Fast Tape set to Off.

Saving your work. A SAVE to a disk is lost when the disk is ejected or the program ends. Use a snapshot, in the F12 menu.

The Keys

Keys	What they do
F11	Disks, tapes and cartridges
F12	The settings
Ctrl+Alt+Del	Reset the CPC
Ctrl+Alt+End	Back to MMBasic
Backspace	DEL
Delete	CLR
Ctrl	CONTROL
Alt	COPY
Caps Lock	CAPS LOCK
Keypad Enter	The small ENTER

The other keys give what is printed on them, following OPTION USBKEYBOARD, so the symbols are where your keyboard has them.

Ctrl-C is a CPC key here, so it does not end the program. USB gamepads are the CPC's joystick.

The Commodore 64 Emulator

This is frank-c64, rh1tech's port of Christian Bauer's Frodo emulator. It is a PAL Commodore 64 with a 1541 disk drive, and runs at the C64's 50 frames a second with most of the processor to spare, in MODE 7.

It runs on the Generation 1 and the Generation 2.

Starting It

```
CHDIR "A:/c64"
RUN "c64.app"
```

A start screen shows for five seconds, and then the C64's BASIC is ready. The C64's ROMs are built into the program.

The program calls the folder it was started in /c64.

Loading Programs

Put disk images (.d64, .g64, .d81), programs (.prg) and cartridges (.crt) in A:/c64, and press **F10**. Choose one with the arrows and Enter, and then:

- **Load (Run)** runs a program or a cartridge. For a disk, it puts the disk in drive 8 and loads the first program on it, with LOAD"*",8,1: type RUN when READY. appears.
- **Mount (Insert)** puts a disk in drive 8 and goes back to BASIC, so that you can type LOAD"$",8 and LIST to see what is on it, then LOAD"NAME",8 and RUN.

Esc goes back a step. Folders inside A:/c64 are listed too, and Enter opens one.

Saving to a disk image, and cartridges, have not been tried yet.

Tapes (.tap, .t64) are listed, but cannot be loaded.

The Keys

Keys	What they do
F1 to F8	The C64's F1 to F8 (its F2, F4, F6 and F8 are Shift with F1, F3, F5 and F7)
F9	Swaps the joystick ports
F10	Disks, programs and cartridges
F11	RESTORE
Ctrl+Alt+Del	Reset the C64
Ctrl+Alt+End	Back to MMBasic
Esc	RUN/STOP
Left Ctrl, Tab	CTRL
Left Alt	The Commodore key
Home	CLR/HOME
Insert	INST
Backspace, Delete	DEL
Caps Lock	SHIFT LOCK
The arrows	The cursor keys

The other keys give what is printed on them, following OPTION USBKEYBOARD, so the symbols are where your keyboard has them. Shift with a letter gives the C64's graphics characters, as on a real C64, or its capitals once Shift and the Commodore key have switched it to lower case. The C64 has keys of its own for some characters: ^ gives its up arrow, _ its left arrow, and \ or £ its pound sign.

Ctrl-C is a C64 key here, so it does not end the program.

Joysticks and the Mouse

The first USB gamepad is joystick port 2, where most games look, and the second is port 1. **F9** swaps them. The d-pad or the left stick steers, and any of the four face buttons fires. A DualShock 3 sends nothing until its PS button has been pressed.

The arrow keys are joystick port 2 as well, with the right Ctrl or right Alt as its fire button.

A mouse is a Commodore 1351 mouse in port 1, for GEOS and the other programs written for one. Its left button is the fire button, and its right button is up.

The Atari 8-bit Emulator

This is atari800, the Atari800 development team's emulator of the Atari 400 and 800, the XL and XE computers and the 5200 console. It starts as an 800XL with 64K, runs at the Atari's own 50 frames a second (PAL), in MODE 7, and has the emulator's own menus.

It runs on the Generation 1 and the Generation 2.

Starting It

```
CHDIR "A:/atari800"
RUN "atari800.app"
```

Words after the name are atari800's own options, for example:

```
RUN "atari800.app", game.xex
RUN "atari800.app", -basic
RUN "atari800.app", -5200 -cart-type 4 -cart game.rom
```

The settings are kept in atari800.cfg in the same folder: the F1 menu's Save Configuration writes it.

The ROMs

AltirraOS and Altirra BASIC, Avery Lee's free replacements for Atari's operating system and BASIC, are built into the program, so it needs nothing else to start, and nearly all programs run with them.

Atari's own ROMs are not included: Atari's copyright in them has not been given up. Copies are in the data folder of DnCraptor's atari800 repository, https://github.com/DnCraptor/atari800 (ATARIXL.ROM, ATARIOSB.ROM and ATARIBAS.ROM), which you download and use at your own risk. Put them in A:/atari800 before the first start: atari800 recognises them by their contents, whatever they are called, and then uses them in place of Altirra's. ROMs added later are found with F1, Emulator Configuration, System ROM settings, Find ROM images in a directory.

BASIC

BASIC is off at first, as atari800 has it, because most programs load into the memory BASIC would use: on a real XL they are started with OPTION held down for the same reason.

Alt+B turns BASIC on or off and starts the Atari again. A BASIC program (.bas) turns it on by itself while it loads. RUN "atari800.app", -basic starts with it on.

Loading Programs

F1 opens the menu, and Alt with a letter goes straight to one of its pages:

Keys	Page
Alt+R	Run Atari Program: a program (.xex, .com), a BASIC program, a disk, a cartridge or a tape
Alt+D	Disk Management: disk images (.atr, .xfd, .atx, .dcm, .pro) in drives D1: to D8:
Alt+C	Cartridge Management: insert or remove a cartridge (.car, .rom)
Alt+T	Tape Management: insert a tape (.cas), its position, recording, a blank tape
Alt+Y	System Settings: the machine, its memory, PAL or NTSC
Alt+J	Controller Configuration: what the mouse is
Alt+S, Alt+L	Save and load the machine's state

Inserting a cartridge starts the Atari again in it, and removing it starts the Atari again without it.

Run Atari Program with a tape (.cas) boots it, as holding START while switching on does. A tape loads at once, as a disk does, because atari800's SIO patch (Emulator Configuration) passes each record straight to the operating system. With the SIO patch off a tape loads at its real speed, 600 baud, with the operating system's tone. RUN "atari800.app", -boottape name.cas boots a tape from MMBasic's prompt.

The Keys

Keys	What they do
F1	The menu
F2, F3, F4	OPTION, SELECT and START

Keys	What they do
F5	RESET (Shift+F5: switch off and on again)
F6	HELP
F7, Pause	BREAK
F9	Back to MMBasic
F12	Turbo: as fast as it can go
Alt+B	BASIC on or off, and the Atari started again
Ctrl+Alt+End	Back to MMBasic
Esc	ESC
Home	CLEAR
Insert, Delete	Insert and delete a character (with Shift, a line)
The arrows	The cursor keys (CTRL with - = + *)
Caps Lock	CAPS
` and the left Windows key	The Atari key (inverse video)

The other keys give what is printed on them, following OPTION USBKEYBOARD, so the symbols are where your keyboard has them. Shift with a letter is a capital, whatever Caps Lock says: Caps Lock is the Atari's own CAPS key, which keeps its own state. Ctrl with a key is the Atari's CTRL with it.

Ctrl-C is an Atari key here, so it does not end the program.

On the 5200 the digits, # (or =) and * are its keypad, P is PAUSE, R is RESET and F4 is START.

Joysticks, Paddles and the Mouse

The first USB gamepad is joystick 1, the one nearly every game uses, and the second is joystick 2. The d-pad or the left stick steers, and any of the four face buttons fires. The pad's Start, Select and L buttons are the console's START, SELECT and OPTION. A DualShock 3 sends nothing until its PS button has been pressed.

With one pad, its right stick is joystick 2, and R1 or R2 its fire, for the twin-stick games such as Robotron: 2084 (this needs firmware with version 16 of the services).

The keypad is joystick 1 as well: 1 to 9 the directions, 0 or 5 the fire.

The mouse can be paddles, an ST or Amiga mouse, a trak-ball, a light pen or gun, or a Koala pad: choose it with Alt+J, Mouse device. With paddles, moving the mouse to the right turns the paddle clockwise, as a real one turns.

Known Problems

- With the SIO patch off, a tape with a program file after its boot loader stops with a checksum error under AltirraOS; with Atari's own XL operating system it loads at real speed. With the SIO patch on, as it starts, both load.
- Programs that need two joysticks on one pad (twin-stick games) need firmware 7.00.00RC4 or later for the pad's right stick.

The Apple //e Emulator

This is frank-apple, rh1tech's port of Michel Pollet's MII emulator. It is an enhanced Apple //e with 128K, an 80-column card, two Disk II drives and two hard disks, and runs at the Apple's 60 frames a second with most of the processor to spare, in MODE 8.

The screen is shown at the Apple's full resolution: 80 columns and double hi-res keep all 560 of their dots across.

It is for the Generation 1 and the Generation 2, but has been tried on a Generation 1 only so far.

Starting It

```
CHDIR "A:/apple2"
RUN "apple2.app"
```

A start screen shows for three seconds, and then the Apple starts. With no disk in a drive it goes to Applesoft BASIC's] prompt. The Apple //e's ROMs are built into the program.

The program calls the folder it was started in /apple.

Disks

Put disk images (.dsk, .do, .po, .nib, .woz) in A:/apple2, and press **F11**:

1. Choose drive 1 or 2 with the arrows (or press 1 or 2), and press Enter.

2. Choose a disk image with the arrows, Page Up and Page Down, and press Enter. Folders are listed too, and Enter opens one.

3. Choose **Boot** to put the disk in and start the Apple from it, or **Insert** to change the disk without starting again, for a program that asks for its next disk. **Cancel** goes back.

Esc goes back a step. In the first step, D takes the disk out of the drive chosen.

The first time a disk image is chosen, a copy of it in the drive's own form is made beside it, with .bdsk added to its name (DISK.DSK.BDSK), and that copy is what the drive uses from then on: the image itself is never changed. A disk is read-only at first, so that a program's changes to it are not kept: Space, in the third step, makes it writable, and the changes then go to its .bdsk file. Tick Recreate .bdsk with D, in the third step, to start again from the image.

Take care in the list of disks: D deletes the file chosen from the card, without asking.

Hard disk images in ProDOS order, up to 32MB, can be put in A:/apple2 as drive0.img and drive1.img. They are the two drives of a SmartPort card in slot 5, which the Apple should start from when there is no disk in the Disk II drives.

Saving to a disk, and the hard disks, have not been tried on the Colour Maximite 2 yet.

The Keys

Keys	What they do
F11	Disks
F12	Runs as fast as it can while held
Scroll Lock	Runs as fast as it can, until pressed again
F9	Shows the speed, at the top of the screen
F10	Changes the colours: NTSC, a second NTSC set, the Apple IIGS's, and a green or amber monitor
Ctrl+Pause	Ctrl+Reset. With the left Alt (Open Apple) as well, the //e starts again from cold
Ctrl+Alt+Del	Starts the Apple again from cold
Ctrl+Alt+End	Back to MMBasic
Left Alt, left Windows key	Open Apple, the first joystick button
Right Alt, right Windows key	Closed Apple, the second joystick button
Backspace, left arrow	The left arrow
Delete	DELETE
Caps Lock	The //e's Caps Lock

The other keys give what is printed on them, following OPTION USBKEYBOARD. The //e's Caps Lock is down at the start, as most programs of the time want capitals: Applesoft and DOS 3.3 do not understand their commands in lower case. Caps Lock lets it up, and then Shift gives the capitals. A key held down repeats, as on the //e.

Ctrl-C is an Apple key here, so it does not end the program: in Applesoft it stops a program.

Joysticks

The first USB gamepad is the Apple's joystick. Its left stick moves it, and the d-pad moves it too, gradually, for the paddle games. Its a and x buttons (circle and triangle on a PlayStation pad) are the first joystick button, b and y (cross and square) the second, and Start the third.

Select opens the disks, where the d-pad moves about, a is Enter and b is Esc. Start, a and b together start the Apple again from cold.

The keypad's 8, 2 (or 5), 4, 6, and 7, 9, 1 and 3 for the diagonals, are the joystick as well, and its 0 the third button.

A DualShock 3 sends nothing until its PS button has been pressed.

The Serial Console

The serial console types into the Apple too, so that a program can be pasted. Return is Return, Backspace is the left arrow, and the Apple's own control keys are its arrows: Ctrl-H left, Ctrl-U right, Ctrl-K up, Ctrl-J down. Ctrl-F then 1 to 9, a, b or c is F1 to F12, so Ctrl-F b gives the disks. Ctrl-F then r is Ctrl+Reset, and Ctrl-] goes back to MMBasic.

RUN "apple2.app", -stats prints, every 5 seconds, the frames a second and how much of the time the emulator has to spare.

The Amiga 500 Emulator

This is an Amiga 500 with 1 MB of memory (512 KB of chip RAM and 512 KB of slow RAM) and its original chipset. It is pico-amiga, drewpo28's emulator for the RP2350, with Karl Stenerud's Musashi 68000, made to run on the Colour Maximite 2. It runs a PAL Amiga at the Amiga's own 50 frames a second, in 720 x 576, with its sound, and the heaviest demos tried keep up.

It runs on the Generation 2 only, and needs firmware 7.00.00RC5 or later: older firmware refuses it with "Needs a newer firmware".

What You Need

The emulator comes without the Amiga's own software. You need two things of your own:

- **A Kickstart ROM**, the Amiga's operating system in ROM. Kickstart is still copyright, so it cannot be given away with the emulator. Kickstart 1.3 is the one the Amiga 500 had and the one to use.
- **Disk images** (.adf files) of the Workbench disk and of the games and demos you want to run.

If you have an Amiga, you can use its Kickstart and disks, saved as files with the usual Amiga tools. If you do not, we recommend **Amiga Forever** from Cloanto (www.amigaforever.com). It is the legal way to get Kickstart and Workbench today, and costs little. Its Value Edition, the least costly, has everything this emulator needs: Kickstart 1.3, the Workbench 1.3 disk, and over 50 games and demos.

Copying the Files from Amiga Forever

Install Amiga Forever on a Windows PC. Its files are then in **C:\\Users\\Public\\Documents\\Amiga Files**. Copy these to the folder **A:/amiga** on the SD card:

Amiga Forever's file	What it is	Name on the card
Shared\\rom\\amiga-os-130.rom	Kickstart 1.3	amiga-os-130.rom (keep its name)
Shared\\adf\\amiga-os-134-workbench.adf	The Workbench 1.3 disk	workbench.adf (or any name)

The emulator looks for **amiga-os-130.rom** in its folder by itself, so keep that file's name exactly as it is.

The other ROMs in Shared\\rom work too, with -kick (below): amiga-os-120.rom is Kickstart 1.2, and amiga-os-110-pal.rom and amiga-os-110-ntsc.rom are Kickstart 1.1. Leave out amiga-os-310-a1200.rom: it is for the Amiga 1200, a later machine with a different processor and chipset, and will not start here.

An older edition of Amiga Forever may have its ROMs locked to it, in files 11 bytes longer than 256 KB. The emulator refuses those with a message giving their size. The current editions' ROMs are plain.

Games and Demos from Amiga Forever

The games and demos are in **Titles\\Games** and **Titles\\Demoscene**, one .rp9 file each. An .rp9 file is a zip file with the title's disk images in it:

1. Copy the .rp9 file somewhere and rename it to end .zip, or open it with 7-Zip.
2. Take out the .adf files. A game on two or more disks has one for each, numbered.
3. Copy them to A:/amiga on the SD card, or to a folder inside it, such as A:/amiga/games.

Only the titles made for the Amiga 500 run here. A title whose zip has a .hdf file (a hard disk) rather than .adf files, or which is marked for the Amiga 1200 or 4000, will not.

Starting It

```
CHDIR "A:/amiga"
RUN "amiga.app"
```

The disk menu comes up, with the disks in A:/amiga. Choose one with the arrow keys and Enter, then **Boot it**, and Kickstart starts it in drive DF0.

To start a disk straight away, name it after the comma, without quotes, with the folder if it is in one:

```
RUN "amiga.app", games/arkanoid.adf
```

The screen stays blank for a few seconds while the disk is read and the display set up.

What the Amiga writes to a disk goes into its .adf file on the card, as it would onto a real floppy disk, so saved games, high scores and your own work last (see Saving Your Work, below).

Disks load about four times as fast as on a real Amiga.

The Disk Menu

Press **F11** at any time, in the middle of a game too, for the disk menu. The Amiga stops while it is up.

The menu lists the disk images (.adf) and the folders in a folder of the card. At first that is the folder of the disk in DF0, or the folder the program started in; after that, the folder you last left it in. The disk in DF0 is marked DF0.

Keys	What they do
Up, Down, Page Up, Page Down, Home, End	Move through the list
A letter or digit	The next name that starts with it
Enter	Open the folder, or choose the disk
Backspace, Left	The folder above
Esc, F11	Back to the Amiga as it was
F12	Back to MMBasic

A disk chosen offers four things:

- **Boot it**: the Amiga restarted, as if it had been switched off and on, with the disk in DF0. Use this to change to another game.
- **Put it in DF0 (a disk change)**: the disk changed, as by hand. Use this when a game asks for its next disk. The drive stays empty for two seconds first, so that the Amiga sees the change.
- **Put it in DF1, the second drive**: the Amiga has an external drive, DF1, as many Amiga 500s had. Put a work disk there, beside the Workbench disk in DF0.
- **Cancel**.

The menu also lists **[A new blank disk, made here]**. Choose it to make an empty disk in the folder shown, ready for the Amiga to use: work1.adf, named Work1 on the Amiga, then work2.adf and so on. The menu then offers to put it in a drive.

A gamepad works the menu too: up and down move, and a face button chooses.

Choices

Add any of these after the disks:

Choice	What it does
-ntsc	An NTSC Amiga, as sold in the USA, at 60 frames a second in 640 x 480
-pal60	The PAL Amiga, shown at 60 frames a second in 640 x 480, for a monitor that does not take 50 Hz. It then runs 20% fast, music too
-nosound	No sound
-kick file	Another Kickstart, such as -kick amiga-os-120.rom
-df1 file	That disk in DF1, the second drive, at the start
-wp	Every disk write-protected: nothing is written to the card

For example:

```
RUN "amiga.app", games/zoom.adf -nosound
```

The Display

The PAL Amiga is shown in 720 x 576 at 50 Hz, the standard 576p picture, with each Amiga line drawn twice. Each Amiga frame is one frame of the monitor, so movement is smooth. Most monitors take this mode. If yours shows it off to one side, try its auto-adjust button, or use -pal60.

Most Amiga games and demos were made in Europe for the PAL Amiga. With -ntsc, such a title shows its picture too low, with its bottom cut off, and some stop altogether, as they did on an NTSC Amiga. Use -ntsc only for titles made for NTSC.

Interlaced screens, with 400 or 512 lines, show every other line for now.

The Keys

The keys are taken by their place on the keyboard, as on an Amiga keyboard.

Keys	What they do
F1 to F10	The Amiga's F1 to F10
F11	The disk menu
F12, or Ctrl+Alt+End	Back to MMBasic
Ctrl+Alt+Del, or Ctrl and both Amiga keys	A reset, as Ctrl and both Amiga keys are on an Amiga: the disk in DF0 starts again
Left Windows key	The left Amiga key
Right Windows key, Menu	The right Amiga key
Alt	The Amiga's Alt
Insert	HELP
Delete	DEL
The arrows, keypad	The Amiga's

Ctrl-C is the Amiga's Ctrl and C here, so it does not end the program.

Workbench 1.3 starts with the American key layout. On a British keyboard, open a Shell and type SetMap gb to have the symbols where your keys show them.

Saving Your Work

What the Amiga writes to a disk goes into the disk's .adf file on the SD card as it is written, as it would onto a real floppy disk, and stays there. This is true of a game's saved positions and high scores, and of anything you save from Workbench or a Shell.

To work on your own programs or files:

1. Start with the Workbench disk in DF0. 2. Press F11, choose **[A new blank disk, made here]**, and put it in DF1. 3. Save your work to it. It is called Work1 on the Amiga, so a file saved as Work1:myprog is in work1.adf.

A disk whose .adf file is read-only (set on a PC) is a write-protected disk: the Amiga says so if it tries to write to it, and the file is never changed. Start the program with -wp to protect every disk, for example to keep a game's disk exactly as it came.

The RAM DISK is in the Amiga's memory, as on a real Amiga, so what is there is lost at a reset or when the program ends.

The Amiga has no BASIC in its ROM. AmigaBASIC came on the Extras disk of Workbench 1.3, which is not among Amiga Forever's Value Edition files.

The Mouse and Joysticks

A USB mouse is the Amiga's mouse, in port 1.

The first USB gamepad is a joystick in port 2, where nearly all games look, and the second is the joystick in port 1. The d-pad or the left stick steers, and any of the four face buttons fires.

Leaving It

Press **F12** or **Ctrl+Alt+End** to go back to MMBasic, in the disk menu too.

What Has Been Tried

On a Generation 2 with firmware 7.00.00RC5, the PAL Amiga in 576p:

- **Demos:** Mental Hangover (Scoopex) and Arte (Sanity), the heaviest tried, at 50 frames a second all through with the sound on, and 242 (Virtual Dreams and Fairlight). D.O.S. (Andromeda) falls just short in a few places.
- **Workbench 1.3**, with the mouse and the keyboard.
- **Games from Amiga Forever**, each for a minute: Arkanoid, Aunt Arctic Adventure, Blasteroids, Datastorm, Exile, Gotcha, Lethal Xcess, Obsession Pinball, Pac-Mania, Rage, Statix, Toobin and Zoom! load and run at 50 frames a second. X-Balls and Logic load slowly through AmigaDOS, and had not finished in that minute. Tie Break shows only a black screen.

- **The keyboard, the mouse and a gamepad**, by hand, in Workbench and in games, and the disk menu.
- **Saving:** a blank disk made from the menu and put in DF1; AmigaDOS wrote a file and a folder to it, and the .adf file, copied from the card to a PC, has them. With -wp the Amiga reported the disk write-protected, and the file was unchanged.

Titles made for later Amigas (the A1200 and A4000), and those on hard disk images, do not run.

Licences

pico-amiga is under the GNU GPL version 3, and Musashi under the MIT licence. Kickstart, Workbench and the games are the property of their owners and do not come with the emulator.

The 386 PC

pc386 is tiny386, Chunhui He's 386 PC emulator, with a VGA card, a Sound Blaster 16, an Adlib and the PC speaker. The Colour Maximite 2's keyboard and mouse are the PC's. It runs FreeDOS, MS-DOS, Windows 3.1 and Windows 95.

This chapter gets you started. Everything else, including installing MS-DOS and Windows, is in its own manual, *Colour Maximite 2 386 PC Emulator*, posted with the firmware.

Starting It

```
CHDIR "A:/pc386"
RUN "pc386.app"
```

The screen changes to 640 x 480, and FreeDOS 1.3 starts from a 32 MB hard disk image. It loads its mouse driver and ends at the C:\> prompt.

A PC is described by a configuration file. The one used unless you say otherwise is pc386.ini. To start a different PC, name its file:

```
RUN "pc386.app", msdos.ini
```

msdos.ini installs MS-DOS 6.2 from images of your own Setup disks, and msdosc.ini then starts it, with Windows 3.1's disks to hand.

Leaving It

Press **Ctrl+Alt+End**. This switches the PC off at once, as if its power had been cut, so first:

- Finish what the PC is doing, and leave Windows if it is running.
- Wait until the PC has stopped writing to its disks.

The PC's disks are image files on the SD card, so everything written to them is kept for the next time.

The Keys

Three keys, held with Ctrl and Alt, are the emulator's own. Every other key goes to the PC, Ctrl+Alt+Delete included: it restarts the PC. Ctrl-C goes to the PC as well, and does not stop the emulator.

Keys	What they do
Ctrl+Alt+End	Leave the emulator
Ctrl+Alt+Page Down	Put the next floppy image of the list in drive A:
Ctrl+Alt+Page Up	Put the previous floppy image in drive A:

The keypad's 1, 3 and 9 work too, if a keyboard does not report those combinations.

DOS uses a US keyboard layout unless you load another. For a UK keyboard, put this in AUTOEXEC.BAT:

```
KEYB UK
```

The Mouse and Sound

DOS programs need a mouse driver. The FreeDOS disk loads CuteMouse, and Windows 3.1 has its own.

Sound comes out of the Colour Maximite 2's audio output. DOS programs find the Sound Blaster at port 220, IRQ 5, DMA 1 (high DMA 5), and an Adlib at port 388. Many of them read these settings from the BLASTER variable, which AUTOEXEC.BAT can set:

```
SET BLASTER=A220 I5 D1 H5 T6
```

Making It Faster

- OPTION SD TIMING FAST makes the SD card, and so the PC's disks, much faster.
- On an original Colour Maximite 2 whose processor is of revision Y, OPTION OVERCLOCK ON makes it as fast as on a Generation 2.

Moving a PC

A PC is a few files in a folder, so copying them moves it to another Colour Maximite 2 of either generation. FreeDOS can be passed on to other people. MS-DOS, Windows and other commercial programs cannot.

The C Compiler

The Colour Maximite 2 can compile programs on the machine itself, with no PC. The CC command compiles a program written in C. It also compiles an MMBasic program, so that a program that spends its time calculating runs much faster.

This chapter covers everyday use. The full details are in its own manual, *Colour Maximite 2 C Compiler*, posted with the firmware.

Installing It

The cc folder has to be at A:/cc. CC looks for it there, whatever the current folder is. It needs firmware 7.00.00RC2 or later.

File	What it is
A:/cc/cc.app	The compiler
A:/cc/include	The header files that C programs include
A:/cc/examples	Two example programs, shapes.c and blink.c

A compiled program needs nothing from A:/cc to run, so you can pass it on by itself.

When It Is Worth Using

Your program	What to use
An MMBasic program that is fast enough	Nothing: leave it as it is
An MMBasic program that spends its time calculating	CC, which is much the fastest way to run it
An MMBasic game that spends its time drawing	Try OPTION COMPILE ON first. CC gains less here, as the drawing takes the same time however the program is run
Something new that has to be as fast as it can be	Write it in C

Compiling an MMBasic Program

Your BASIC program is not changed. RUN "game.bas" still runs it with the interpreter, and the compiled copy sits beside it in a file of its own. Change to the program's folder first, as a game finds its pictures and sounds from there:

```
CHDIR "A:/game"
CC "game.bas"
RUN "game.b.bc"
```

CC prints a row of dots as it works. A program of about 3,000 lines compiles in 7 seconds. The largest tried so far, 4,300 lines, takes about a minute and a half, and needs a Generation 2.

If you change game.bas, run CC again. RUN "game.b.bc" does not check that it is up to date.

Read What CC Reports

If CC cannot translate a statement it does not stop. **The statement is left out**, and CC reports the line and the reason. Nothing runs in its place, so a program with a line left out may run and do the wrong thing. Read the report, particularly if the line left out opened a block.

Check the Result

A compiled program should behave as the original does, but yours may do something that has not been tried. Run the BASIC program and the compiled one, and check that they agree before you rely on it.

A compiled program is faster, so a game that paces itself by how long its own code takes will run too fast. It needs a timer in its main loop.

Compiling a C Program

Type this into a file called hello.c, with EDIT or on a PC:

```
#include <stdio.h>

int main(void)
{
    printf("Hello from C, %d\n", 6 * 7);
    return 0;
}
```

Then, in the same folder:

```
CC "hello.c"
RUN "hello.bc"
```

CC makes hello.bc beside hello.c, in about a second. MMBasic carries on when the program ends, and Ctrl-C stops a program as it does in BASIC.

RUN can do both steps. RUN "hello.c" compiles the file if hello.bc is missing or older than it, and then runs it, so editing a program and trying it again is one step.

Words after a comma are the program's arguments, in argv[1] onward:

```
RUN "hello.bc", one two
```

The Language

The language is C89 (ANSI C) with long long, // comments and declarations after statements. A program is one source file, as there is no linker. Bit fields are not supported, nor are declarations inside a for statement.

The usual stdio.h, stdlib.h, string.h, math.h, ctype.h and time.h are there. There is no scanf. Files have MMBasic's names, such as A:/dir/name.txt.

cmm2.h gives a C program the Colour Maximite 2's own facilities: the video modes, drawing with MMBasic's own routines, the keyboard, the mouse, sound and the pins:

```
#include <cmm2.h>

int main(void)
{
    cmm2_video v;

    video_mode(1, 8, &v);
    gfx_cls(BLACK);
    gfx_circle(400, 300, 100, 2, 1.0, GREEN, RED);
    gfx_text(400, 300, "Hello", "CM", 1, 2, WHITE, NOFILL);
    return 0;
}
```

CC's Options

CC takes a second argument, a string of options:

Option	What it does
-v	Shows each pass of the compiler as it runs
-k	Keeps the intermediate files
-o name	Gives the object file another name

For example:

```
CC "hello.c", "-o greet.bc"
```

How Much Faster

A 3,200-line calculation, on a Generation 2 at 480MHz:

How it ran	Time	Compared with the interpreter
The interpreter	6.16 seconds	1
OPTION COMPILE ON	1.71 seconds	3.6 times faster
CC	0.93 seconds	6.6 times faster

Games gain less, as they spend much of their time drawing. Try OPTION COMPILE ON first: it needs no extra step and works on any program.

When Things Go Wrong

What you see	What to do
CC cannot find the compiler	cc.app has to be in A:/cc. Copy the cc folder from the zip to the card, and check its name
Cannot open include file stdint.h	The files in A:/cc/include are from an older version. Copy the whole cc folder from the zip that came with your firmware
x.c did not compile	The lines before it give the file, line number and error. A missing semicolon is reported on the next line
A compiled BASIC program does the wrong thing	Look at what CC reported. A line it could not translate is left out
A game runs too fast	It needs a timer in its main loop, as the compiled program is faster
not enough memory for its tables, jump out of range, code segment full	The program is too large for the compiler on a Generation 1. Compile it on a Generation 2
The compiled program cannot find its files	CHDIR to the program's folder before RUN

The VT100 Terminal

This program makes the Colour Maximite 2 a terminal: the screen and keyboard of another computer. Plug a PicoMite into it and you can use the PicoMite's prompt, its editor and its file manager, in colour, with the mouse, without a PC.

It has a setup menu, scrollback, a log, and XMODEM for moving files. It is a VT100 and VT102, with xterm's 256 colours and mouse reports added, and VT52 mode. It uses libvterm, with the Spleen fonts.

It needs firmware 7.00.00RC3 or later for USB serial devices.

Connecting

Port	What it is
COM4 to COM7	USB serial devices, such as a PicoMite or a USB serial adapter, in a USB socket or through a hub
COM1	The serial port on GP0 and GP1 of the rear connector
COM2	The serial port on GP36 and GP37 of the rear connector

The rear connector's serial ports use 3.3 V signals and are not 5 V tolerant. Do not connect them to an RS-232 port.

When a USB serial device is plugged in, the MMBasic prompt shows which port it is, starting at COM4.

Starting It

```
CHDIR "A:/vt100"
RUN "vt100.app", COM4 115200
```

The port and speed can be left out once they are saved in the setup menu. Any setting can be given on RUN's line by name, such as font=3.

The line at the bottom of the screen shows the port, its speed and the size of the terminal.

Leaving It

Press **Shift-F12**, or Pause, to open the setup menu, then choose **Quit**.

Ctrl-C does not leave the terminal. It goes to the computer at the other end, as it should, to stop a program there.

Using It With a PicoMite

Everything works as it does on a terminal on a PC:

- The PicoMite's prompt, EDIT and FM work with the arrow keys, the function keys and the mouse.
- When the PicoMite asks for a screen size, at EDIT, FM and OPTION DISPLAY, the terminal changes to it. It tries a smaller font in the same video mode first, so your monitor rarely has to lock on again.
- Colours are shown as the PicoMite sends them.

The block on the screen in EDIT is the mouse pointer, and the underline is the text cursor.

If the PicoMite Restarts

Many PicoMite OPTION commands restart it, and it also drops off when you unplug it. You do not have to do anything: the status line says **COM4 disconnected, waiting for it to come back**, and when the device returns the terminal carries on by itself.

The Keys

Keys	What they do
Shift-F12, or Pause	The setup menu
Shift with Page Up, Page Down	Look back through what has scrolled off the screen
Shift with Home, End	The top of the scrollback, and back to the live screen
The mouse wheel	Scrollback too, unless the other computer is using the mouse
Any other key	Goes back to the live screen, and to the other computer
Ctrl-J	Sends a line feed, on its own

The Setup Menu

Up and Down choose a setting, and Left, Right or Enter change it. Esc goes back to the terminal. Choose **Save settings** to keep your changes, which go in vt100.cfg in the current folder.

Setting	What it does
Port, Speed, Format	The connection. Format is the data bits, parity and stop bits
Screen	The video mode
Font	From 5 x 8 to 16 x 32. 8 x 14 is the default
Columns, Rows	The terminal's size. All uses the whole screen
Status line	The line at the bottom
Characters	How bytes above 127 are shown: CP437 (as a PicoMite has them), Latin-1 or UTF-8
Text colour	White, green or amber, like a monitor of the time
Cursor	Underline, Block or Bar
Backspace sends	BS (8) or DEL (127)
Enter sends	CR, or CR LF. A PicoMite wants CR
LF received as	LF, or CR LF. This is for what arrives, and does not change what your keys send
Local echo	Shows what you type, for a computer that does not send it back
Text line delay	The pause after each line when sending a text file
Send text file...	Sends a file as if you had typed it
XMODEM send..., XMODEM receive...	Moving files (below)
Log to file...	Records everything that arrives in a file
Reset the terminal	Puts the terminal back as it starts
Save settings	Keeps these settings
Quit	Leaves the terminal

Moving Files With XMODEM

Files can be moved each way, between the SD card and a PicoMite or other computer.

From the Colour Maximite 2 to a PicoMite

1. At the PicoMite's prompt, type XMODEM RECEIVE "name.txt" and press Enter.
2. Straight away press Shift-F12, choose **XMODEM send...** and press Enter.
3. Pick the file from the list and press Enter.

The PicoMite waits only a limited time, so go straight on from step 1 to step 2.

From a PicoMite to the Colour Maximite 2

1. At the PicoMite's prompt, type XMODEM SEND "name.txt" and press Enter.
2. Press Shift-F12, choose **XMODEM receive...** and press Enter.
3. Type a name for the file and press Enter.

Esc stops a transfer. Retries in the progress display mean the line is having trouble, even if the file arrives whole.

File Sizes

XMODEM sends files in blocks of 128 bytes with no length, so the last block is padded. A text file arrives exactly as it was. A binary file keeps the padding, and can arrive up to 127 bytes longer. Nothing is lost.

zeptoforth

zeptoforth is Travis Bemann's Forth for ARM microcontrollers, at https://github.com/tabemann/zeptoforth. On the Colour Maximite 2 it is a complete Forth system with preemptive tasks, double-precision arithmetic, a screen text editor, and words for the screen, sound, files and I/O pins.

Why Forth

Forth is a very small language that you extend as you go. A program is a set of words. You build each word from the words you already have, and from then on it is part of the language, no different from the words that came with it. Numbers pass between words on a stack, so a word needs no declarations and no argument lists:

```
: square  dup * ;
7 square .
```

The first line defines a word that squares a number. The second puts 7 on the stack, squares it, and prints 49.

Forth is compiled and interactive at once. Each word becomes ARM machine code the moment you finish typing it, on the Colour Maximite 2 itself, with no PC and no build step, and you can try it at the prompt straight away. You build a program from the bottom up, testing each small word as you go, so a fault is usually in the last thing you typed.

The language is yours to change. The compiler is made of words too, so you can add control structures of your own or change how numbers are read. And nothing stands between you and the machine: words read and write any address, and there is an assembler when you need one.

Starting It

Copy the zeptoforth folder to the card, then on a Generation 2:

```
CHDIR "A:/zeptoforth"
RUN "zeptoforth.app"
```

On a Generation 1:

```
CHDIR "A:/zeptoforth"
RUN "zeptoforth_g1.app"
```

Each has its own saved Forth system, zeptoforth.img or zeptoforth_g1.img, in the same folder. Keep them together.

Leaving It

Type **bye** to go back to MMBasic.

Ctrl-C stops a word that is running and starts Forth again. What you compiled to RAM is lost, and what you saved is kept. Ctrl-C does not leave zeptoforth.

The Console

The console is a terminal on the screen, the same one as the VT100 terminal program, so Forth's colours and editors work on the Colour Maximite 2 with no PC. It is also sent to the serial port, and you can type at either.

Shift with Page Up and Page Down, or the mouse wheel, looks back through what has scrolled off the screen.

The font suits the video mode. To change it, or the size of the console:

```
cmm2 import
4 console-font
80 24 console-size
```

console-font takes 0 to 5, from 5 x 8 to 16 x 32, or -1 to go back to the one that suits the mode. console-size takes the columns and lines, and 0 0 means all there is.

To recall and edit what you have typed with the arrow keys, type **enable-line**. Up and Down go through the lines you typed before. disable-line turns it off, as you should before sending a file to the console from a PC.

Programs that ask how big the terminal is get the size of the screen, so a terminal program on a PC should be set to the same size, or use console-size.

Editing Files

zeptoed is a full-screen editor for files on the SD card or USB drive:

```
s" A:/zeptoforth/mycode.fs" zed
```

The file is made if it is not there. Names are MMBasic's, with the drive and folders, or relative to the current folder.

Key	What it does
Ctrl-W	Saves the file
Ctrl-V	Leaves the editor
Ctrl-Z	Undoes the last change
Ctrl-R	Searches forward
Ctrl-Space	Starts or clears a selection
Ctrl-K	Cuts the selection
Ctrl-Y	Pastes
Ctrl-O	Opens another file
Ctrl-N, Ctrl-P	Moves between the files open, and the help
Ctrl-? or Ctrl-/	Shows all the keys. Ctrl-P goes back to your file, and Esc then Ctrl-X closes the help

Files in the editor are held in 512 KB of memory, with what it needs to undo your changes. To change it, type for example 1000000 to zeptoed-heap-size.

Ctrl-C restarts Forth from the editor as it does anywhere, so save first.

Loading a Program

```
include mycode.fs
```

A file may include another, up to four deep. fat32-tools import gives zeptoforth's own file words, such as list-dir, list-file and copy-file. The module cmm2 has MMBasic's, such as files and chdir.

Keeping Your Work

Words go to RAM to begin with, and are lost at a restart. To keep them, compile them to the flash dictionary and save it:

```
compile-to-flash
include mycode.fs
compile-to-ram
save-image
```

save-image writes the whole Forth system back to zeptoforth.img or zeptoforth_g1.img, and it is there at the next start. What zeptoforth calls flash is memory here, so nothing is written to the Colour Maximite 2's own flash.

Keep a copy of the image file before you save over it. If a saved image stops zeptoforth from starting, put the copy back.

Blocks

Forth's blocks are kept in the file blocks.dat in the current folder, up to 4096 blocks of 1024 bytes. **n edit** opens block n in the block editor, Ctrl-V leaves it, and **n list** and **n load** work as they do in any Forth.

Graphics, Sound and I/O

The words for the screen, sound, the keyboard and mouse, files and the I/O pins are in the module cmm2:

```
cmm2 import
1 8 mode
300 300 80 2 1de0 cyan orange circle
```

WORDS.md in the folder lists them. Their arguments are in MMBasic's order. The console goes on working in any video mode, on top of what you draw. A graphics program should usually turn it off the screen with 0 console-screen, so that nothing it prints lands on the picture.

Double Precision

Decimal numbers have a d before the exponent, and the words have dv in front:

```
float64 import
2de0 dvsqrt dv.
```

The words are those of zeptoforth's single-precision float32, with v changed to dv: dv+, dvsin, dv. and so on. A value takes two cells. A program written with the names v+ and numbers like 1.5e0 can type include float64_v.fs and float64-v import.

Learning More

WORDS.md in the folder describes every word this version adds: the screen, sound, files and I/O, double precision, blocks and the rest. zeptoforth's own reference has everything else, at https://github.com/tabemann/zeptoforth/tree/master/docs/words.

If Forth is new to you, these are good places to start:

Starting Forth	**Leo Brodie's classic introduction, free at https://www.forth.com/starting-forth/**
Easy Forth	A short tutorial that runs in your browser, at https://skilldrick.github.io/easyforth/
Thinking Forth	Leo Brodie's book on how to design programs in Forth, free at https://thinking-forth.sourceforge.net

Forths differ in their details, so a word in a book may have another name here, or need a module imported.

Errors

A bad address gives the message **bad address**, and you go back to the prompt with what is in RAM kept. Inside a firmware service it restarts Forth instead.

If a Generation 1 says **the memory it is linked for is not free**, MMBasic is using too much of its memory. Run zeptoforth from the prompt after a restart, with no program running.

Memory

	Generation 2	**Generation 1**
RAM dictionary and stacks	3 MB	1 MB
Flash dictionary	2 MB	768 KB

The Forth system itself takes about 510 KB of the flash dictionary.

Appendix A: Licences

The programs have their own licences, which are in the licences folder of CMM2_Programs_7.00.00RC5.zip. README.txt in the same zip gives each in full, with the permissions for the ROMs. Keep the licences folder with the programs if you pass them on.

The source of each program is named in the file make.py of its folder in the firmware's repository, https://github.com/UKTailwind/CMM2V1.5, with the changes made for the Colour Maximite 2 beside it.

Program	Licence
MicroPython	MicroPython, ulab and usqlite: MIT. SQLite: public domain
Doom	GNU GPL version 2. Freedoom: BSD 3-Clause
Heretic	GNU GPL version 2. The shareware game data is copyright Raven Software and id Software, under its own licence (LICENSE.DOC): it is not free software and may not be sold
Quake	GNU GPL version 2. The shareware game data is copyright id Software, under its own licence (SLICNSE.TXT): it is not free software and may not be sold
ScummVM	GNU GPL version 2 or later, with parts under other licences as ScummVM's files say
NES	frank-nes: GNU GPL version 3. QuickNES: GNU LGPL version 2.1 or later. Nova the Squirrel: code GNU GPL version 3 or later, graphics and levels Creative Commons BY-NC-SA 4.0. Thwaite: GNU GPL version 3 or later
ZX Spectrum	pico-spec: GNU GPL version 3. miniz and the WD1793 emulation in it: MIT. The ROMs are not under the GPL (see README.txt)
Amstrad CPC	frank-cpc: GNU GPL version 3, with Caprice32 and cpc4x under GNU GPL version 2 or later. The ROMs are not under the GPL (see README.txt)
Commodore 64	frank-c64 and Frodo: GNU GPL version 2 or later. The ROMs are not under the GPL (see README.txt)
Atari 8-bit	atari800: GNU GPL version 2 or later. AltirraOS and Altirra BASIC, built in: copyright Avery Lee, and free to copy and pass on with their notice (see README.txt)
Apple //e	frank-apple and MII: MIT. The ROMs are Apple's, and not under the MIT licence (see README.txt)
Amiga 500	pico-amiga: GNU GPL version 3. Musashi: MIT. No Kickstart, Workbench or other Amiga software comes with it
386 PC	tiny386: BSD 3-Clause. Its devices: MIT. inih: BSD 3-Clause. fmopl: GNU LGPL version 2.1 or later. SeaBIOS and SeaVGABIOS: GNU LGPL version 3. FreeDOS: each program under its own licence, most of them the GNU GPL
C compiler	The Fuzix Compiler Kit: GNU GPL version 3. The translator from MMBasic to C: the MMBasic licence
VT100 terminal	libvterm: MIT. The Spleen fonts: BSD 2-Clause
zeptoforth	zeptoforth: MIT. Its console has libvterm: MIT, and the Spleen fonts: BSD 2-Clause